

Mobile User Behavior Analysis

Objective

Perform exploratory data analysis (EDA) and build machine learning models to predict User Behavior Class based on mobile usage patterns.

Tools Used

- Python
- Pandas
- NumPy
- Matplotlib & Seaborn
- Scikit-learn

Author

Prakhar Goel

✓ Introduction

This project analyzes mobile user behavior data to understand how users interact with mobile applications. The aim is to identify patterns such as screen time, app usage, and user engagement.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier

sns.set_style("whitegrid")

print("Libraries Loaded ")
```

Libraries Loaded

```
df = pd.read_csv("user_behavior_dataset.csv")

df = df.drop("User ID", axis=1)

df.head()
```

	Device Model	Operating System	App Usage Time (min/day)	Screen On Time (hours/day)	Battery Drain (mAh/day)	Number of Apps Installed	Data Usage (MB/day)
0	Google Pixel 5	Android	393	6.4	1872	67	1122
1	OnePlus 9	Android	268	4.7	1331	42	944
2	Xiaomi Mi 11	Android	154	4.0	761	32	322
3	Google Pixel 5	Android	239	4.8	1676	56	871

Next steps:

[Generate code with df](#)[New interactive sheet](#)

✓ Initial Data Exploration

Before preprocessing, I explored the dataset to understand available features and sample records.

```
df.columns
```

```
Index(['Device Model', 'Operating System', 'App Usage Time (min/day)',
      'Screen On Time (hours/day)', 'Battery Drain (mAh/day)',
      'Number of Apps Installed', 'Data Usage (MB/day)', 'Age', 'Gender',
      'User Behavior Class'],
      dtype='object')
```

```
df.sample(5)
```

	Device Model	Operating System	App Usage Time (min/day)	Screen On Time (hours/day)	Battery Drain (mAh/day)	Number of Apps Installed	Data Usage (MB/day)
614	Samsung Galaxy S21	Android	106	4.0	1158	23	4
79	Google Pixel 5	Android	230	5.7	1254	52	9
55	iPhone 12	iOS	68	1.6	450	14	
123	Google Pixel 5	Android	529	8.1	2686	96	19

Dataset Overview

We inspect the dataset structure, column names, and statistical summary before preprocessing.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700 entries, 0 to 699
Data columns (total 10 columns):
 #   Column                                  Non-Null Count  Dtype  
---  -
 0   Device Model                          700 non-null   object 
 1   Operating System                      700 non-null   object 
 2   App Usage Time (min/day)              700 non-null   int64  
 3   Screen On Time (hours/day)            700 non-null   float64 
 4   Battery Drain (mAh/day)               700 non-null   int64  
 5   Number of Apps Installed              700 non-null   int64  
 6   Data Usage (MB/day)                  700 non-null   int64  
 7   Age                                   700 non-null   int64  
 8   Gender                                700 non-null   object 
 9   User Behavior Class                  700 non-null   int64  
dtypes: float64(1), int64(6), object(3)
memory usage: 54.8+ KB
```

```
df.describe()
```

	App Usage Time (min/day)	Screen On Time (hours/day)	Battery Drain (mAh/day)	Number of Apps Installed	Data Usage (MB/day)	Age
count	700.000000	700.000000	700.000000	700.000000	700.000000	700.00000
mean	271.128571	5.272714	1525.158571	50.681429	929.742857	38.48285
std	177.199484	3.068584	819.136414	26.943324	640.451729	12.01291
min	30.000000	1.000000	302.000000	10.000000	102.000000	18.00000
25%	113.250000	2.500000	722.250000	26.000000	373.000000	28.00000
50%	227.500000	4.900000	1502.500000	49.000000	823.500000	38.00000
75%	434.250000	7.400000	2229.500000	74.000000	1341.000000	49.00000
max	700.000000	10.000000	3000.000000	100.000000	2100.000000	70.00000

```
df.isnull().sum()
```

	0
Device Model	0
Operating System	0
App Usage Time (min/day)	0
Screen On Time (hours/day)	0
Battery Drain (mAh/day)	0
Number of Apps Installed	0
Data Usage (MB/day)	0
Age	0
Gender	0
User Behavior Class	0

```
dtype: int64
```

✓ Missing Value Analysis

No missing values were found in the dataset.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == "object":
        df[col] = le.fit_transform(df[col])
```

```
df.head()
```

	Device Model	Operating System	App Usage Time (min/day)	Screen On Time (hours/day)	Battery Drain (mAh/day)	Number of Apps Installed	Data Usage (MB/day)
0	0	0	393	6.4	1872	67	1122
1	1	0	268	4.7	1331	42	944
2	3	0	154	4.0	761	32	322
3	0	0	239	4.8	1676	56	871
4	4	1	187	4.3	1367	58	988

Next steps:

[Generate code with df](#)[New interactive sheet](#)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
X = df.drop("User Behavior Class", axis=1)
y = df["User Behavior Class"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
print("Training samples:", X_train.shape)
print("Testing samples:", X_test.shape)
```

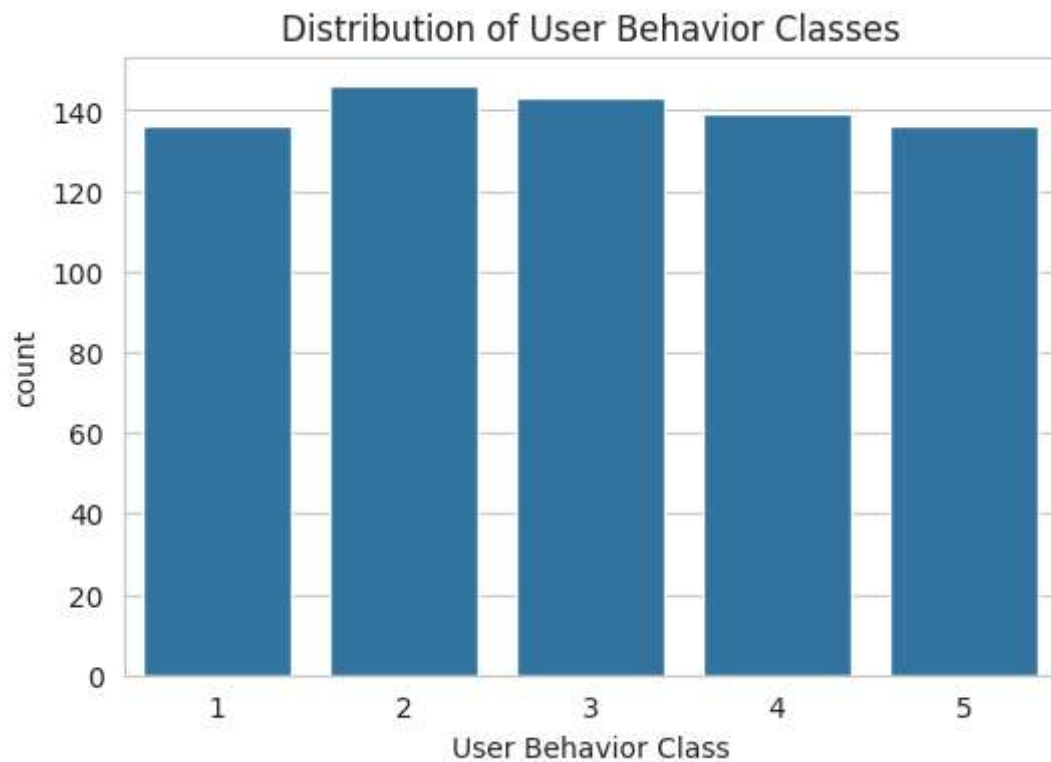
```
Training samples: (560, 9)
Testing samples: (140, 9)
```

✓ User Behavior Class Distribution

We analyze how user behavior classes are distributed in the dataset.

```
plt.figure(figsize=(6,4))
sns.countplot(x='User Behavior Class', data=df)

plt.title("Distribution of User Behavior Classes")
plt.show()
```



Insight

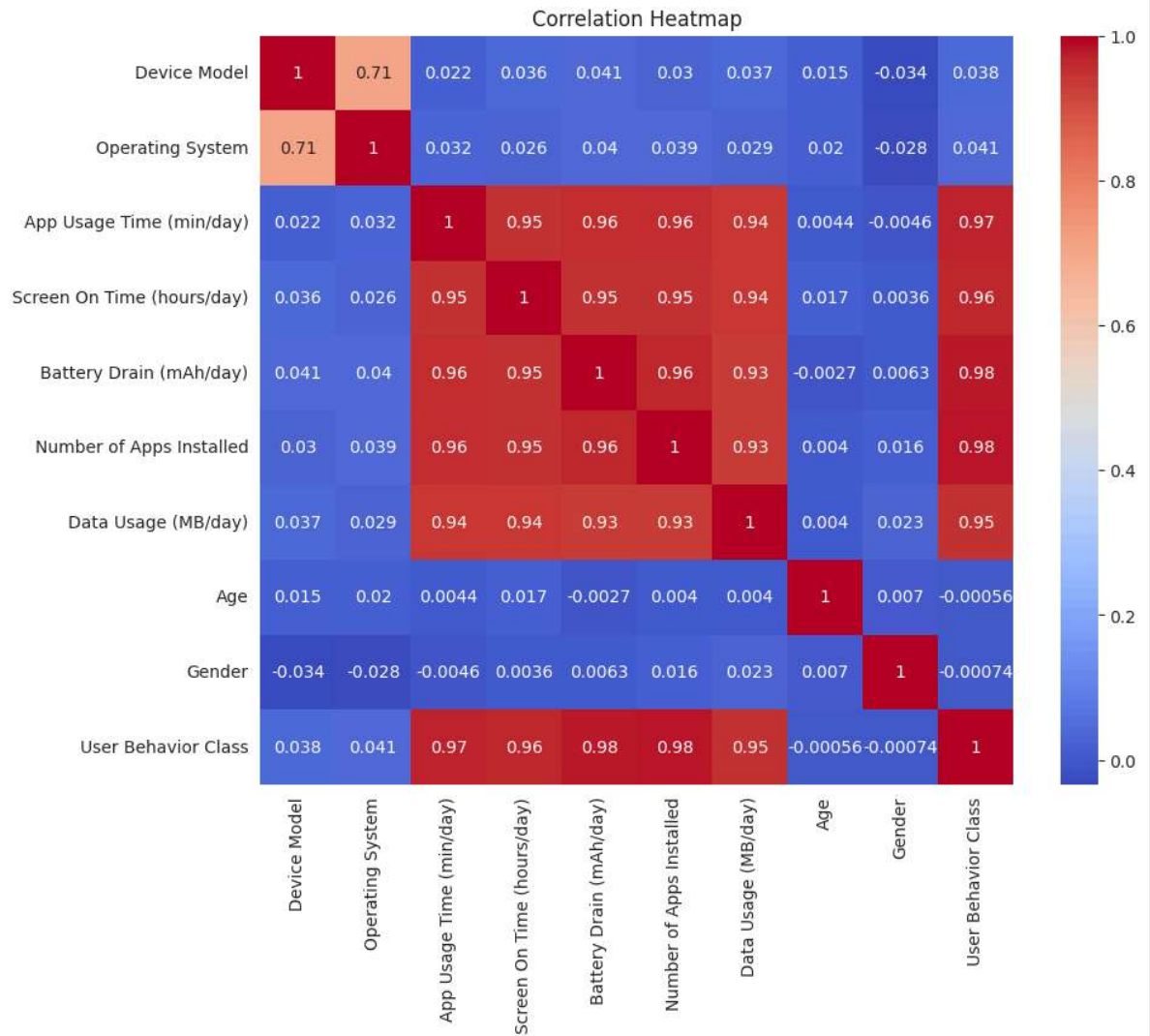
From the graph, I noticed that users who spend more time on apps usually fall into higher behavior classes. This suggests that heavy phone usage directly affects user behavior patterns.

✓ Feature Correlation Heatmap

We examine relationships between numerical features.

```
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")

plt.title("Correlation Heatmap")
plt.show()
```



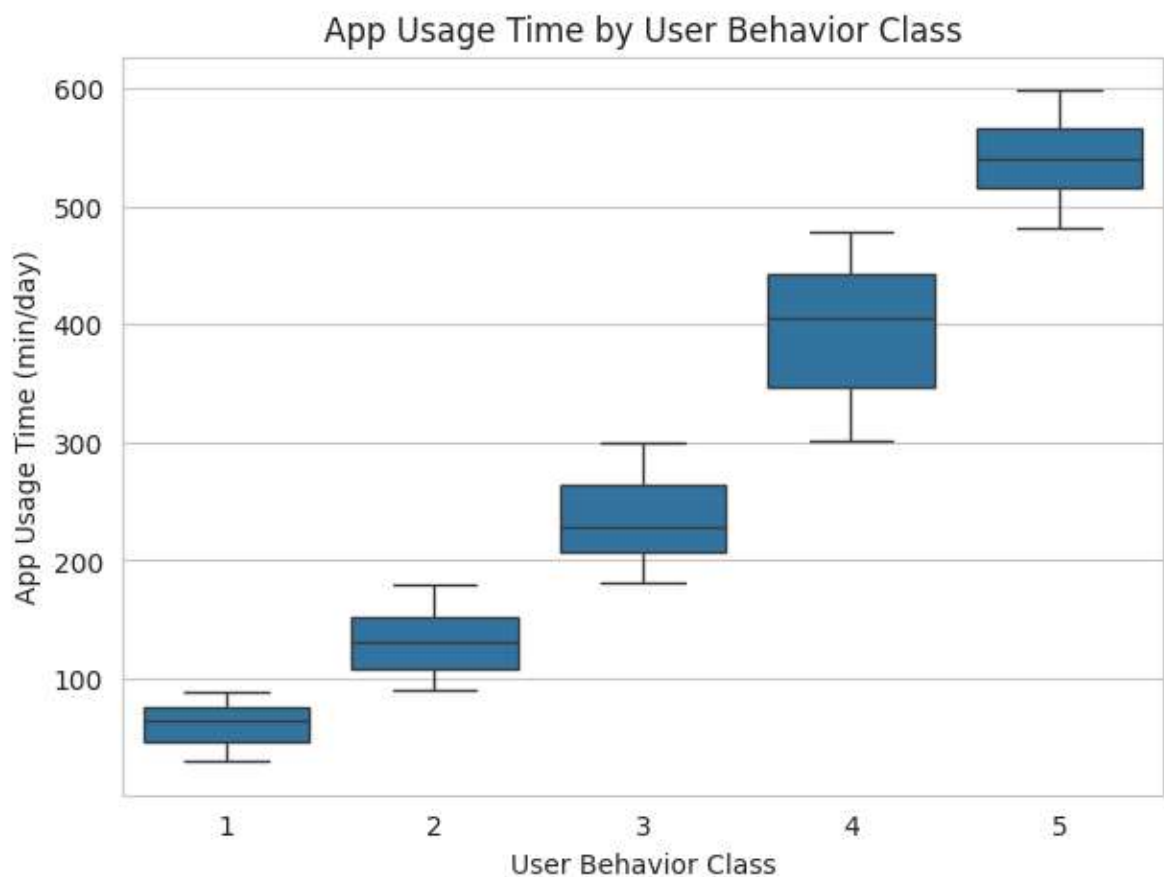
Insight

Screen-on time, app usage time, and battery drain appear strongly correlated, indicating that increased device usage directly impacts battery consumption.

✓ App Usage Time vs User Behavior Class

```
plt.figure(figsize=(7,5))
sns.boxplot(x='User Behavior Class', y='App Usage Time (min/day)', data=df)

plt.title("App Usage Time by User Behavior Class")
plt.show()
```



Insight

Higher behavior classes tend to show increased app usage time, suggesting heavier engagement with mobile devices.

Model Building

A machine learning model is used to predict user behavior.

✓ Model 1: Logistic Regression

We train a Logistic Regression model to predict user behavior classes.

```
log_model = LogisticRegression(max_iter=1000)

log_model.fit(X_train, y_train)

y_pred_log = log_model.predict(X_test)

log_accuracy = accuracy_score(y_test, y_pred_log)

print("Logistic Regression Accuracy:", log_accuracy)
```

```
Logistic Regression Accuracy: 1.0
```

✓ Model 2: Decision Tree Classifier

Decision Trees capture nonlinear relationships between features.

```
dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)

dt_accuracy = accuracy_score(y_test, y_pred_dt)

print("Decision Tree Accuracy:", dt_accuracy)
```

```
Decision Tree Accuracy: 1.0
```

✓ Model 3: K-Nearest Neighbors (KNN)

KNN classifies users based on similarity to nearby samples.

```
knn_model = KNeighborsClassifier(n_neighbors=5)

knn_model.fit(X_train, y_train)

y_pred_knn = knn_model.predict(X_test)

knn_accuracy = accuracy_score(y_test, y_pred_knn)

print("KNN Accuracy:", knn_accuracy)
```

```
KNN Accuracy: 0.9928571428571429
```

Model Evaluation

The performance of the model is evaluated using accuracy score and classification report.

✓ Model Performance Comparison

```
results = pd.DataFrame({  
    "Model": ["Logistic Regression", "Decision Tree", "KNN"],  
    "Accuracy": [log_accuracy, dt_accuracy, knn_accuracy]  
})
```

results

	Model	Accuracy	
0	Logistic Regression	1.000000	
1	Decision Tree	1.000000	
2	KNN	0.992857	

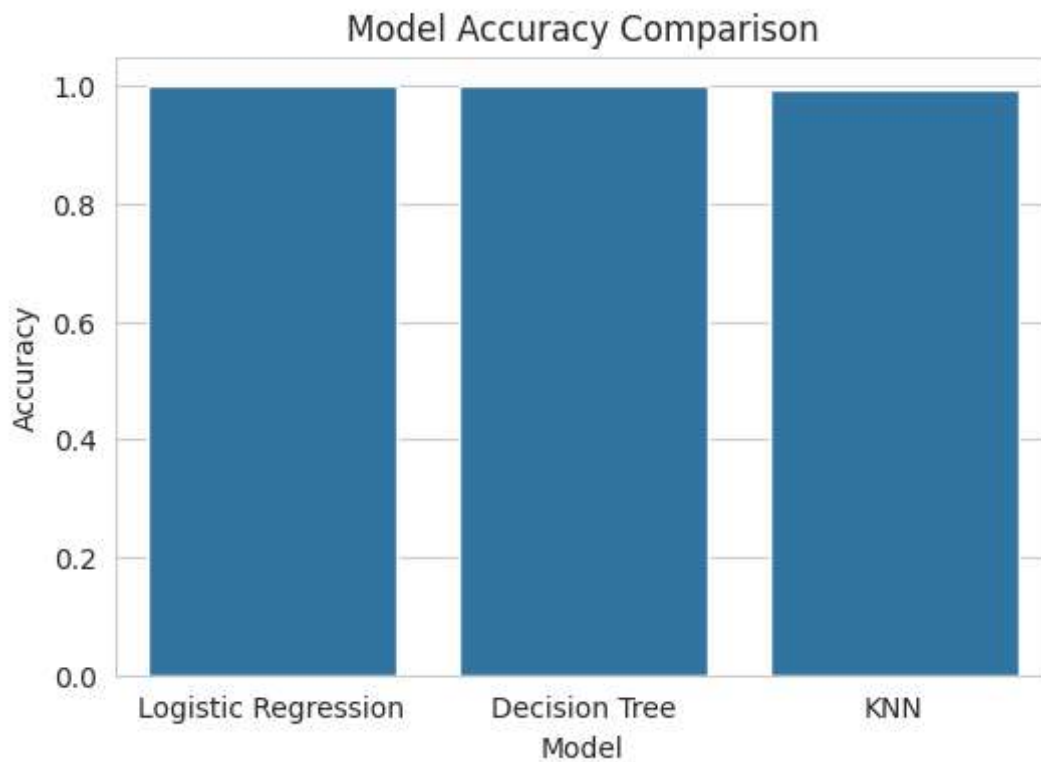
Next steps:

[Generate code with results](#)[New interactive sheet](#)

```
print(results.sort_values(by="Accuracy", ascending=False))
```

	Model	Accuracy
0	Logistic Regression	1.000000
1	Decision Tree	1.000000
2	KNN	0.992857

```
plt.figure(figsize=(6,4))  
sns.barplot(x="Model", y="Accuracy", data=results)  
  
plt.title("Model Accuracy Comparison")  
plt.show()
```



✓ Confusion Matrix (Best Model)

```
best_model_pred = y_pred_knn # change if another model is better

cm = confusion_matrix(y_test, best_model_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Confusion Matrix